Regresión Lineal con Gradiente Descendiente en Julia

Implementación Académica con Interfaz Nativa, Cuaderno Reactivo y Comparación con Biblioteca

S. Rojas

8vo Semestre — Aprendizaje Automático

2026

Resumen

Se presenta una implementación didáctica de regresión lineal univariada con gradiente descendiente en el lenguaje Julia. El trabajo se organiza en torno a dos programas principales: una aplicación de escritorio con interfaz nativa Gtk (`lineal_regresion_guinativa.jl`) y un cuaderno reactivo de Pluto.jl (`regresion_pluto.jl`). Ambos construyen explícitamente el modelo, la función de costo, las derivadas parciales y la regla de actualización simultánea, sin recurrir a bibliotecas de aprendizaje automático. Como referencia profesional se incluye una tercera versión (`regresion_lineal_libreria.jl`) que resuelve el mismo problema con la biblioteca `GLM.jl`, mostrando que una función de biblioteca equivale, en la práctica, a fijar un conjunto de hiperparámetros ya validados por otro. Para facilitar la comprensión se emplea la analogía de una radio antigua que debe sintonizarse girando perillas, haciendo tangibles los símbolos matemáticos.

1. Introducción

La regresión lineal es el punto de partida del aprendizaje automático: modelo parametrizado, función de costo y un algoritmo de optimización que ajusta los parámetros para reducir el error [1]. Entender este caso simple es entender el esqueleto que sostiene modelos mucho más grandes [4].

Este reporte documenta dos implementaciones complementarias escritas en Julia que resuelven el problema paso a paso, sin abstracciones: una con interfaz nativa Gtk y otra como cuaderno reactivo en Pluto. Una tercera implementación, basada en la biblioteca `GLM.jl`, se incluye como ejemplo de cómo luce el mismo problema resuelto por una biblioteca estadística profesional; su interés aquí es pedagógico, ya que permite ver que los hiperparámetros que el usuario ajusta a mano en las dos primeras versiones son equivalentes a las decisiones que la biblioteca toma internamente de forma automática.

El objetivo es doble: construir un programa funcional donde el estudiante pueda modificar parámetros y observar el efecto en tiempo real, y hacerlo con código lo suficientemente transparente para que cada línea se corresponda con un símbolo de la formulación matemática.

1.1. Analogía de la radio antigua

Antes de ir al formalismo, conviene una imagen concreta. Imagínese una radio muy antigua rescatada del ático. Se conecta y al principio solo sale ruido por la bocina; el objetivo es sintonizar

una estación que sí se escucha con claridad. Bajo esa idea cada símbolo tiene un correlato físico:

- x (entradas): las ondas electromagnéticas que chocan contra la antena. Es información del entorno que no se controla.
- y (salida real): la canción original, tal como la emite la estación. Es la meta.
- $f(x) = wx + b$ (predicción): el sonido que efectivamente sale por la bocina en este instante.
- J (función de costo): el dolor de cabeza de escuchar estática. Mide qué tan lejos está el sonido real de la canción deseada. El objetivo es reducirlo a cero.
- m (número de datos): las frecuencias distintas que se prueban para asegurarse de que la radio funciona en general, no solo en un canal afortunado.
- w (peso o pendiente): la perilla principal de sintonia. Afecta directamente cómo se traducen las ondas que entran en sonido.
- b (sesgo o intercepto): el ajuste fino o la posición de la antena. Sube o baja el nivel base de la señal para quitar un zumbido constante.
- $\partial J/\partial w$, $\partial J/\partial b$ (gradientes): el oído del técnico. Dicen en qué dirección girar cada perilla para que el ruido baje en lugar de subir.
- α (tasa de aprendizaje): la brusquedad con la que se gira la perilla. Si la mano es pesada, se pasa de la estación; si es demasiado débil, se tarda toda la noche.
- Número de iteraciones: cuántos empujoncitos de paciencia se le dan a la perilla antes de parar.

El programa es, en esencia, un robót ciego que usa las derivadas como oídos para girar w y b con fuerza α, repitiendo hasta que la canción suene lo más limpia posible.

2. Metodología

Esta sección describe los fundamentos matemáticos del algoritmo y la estructura del código implementado. Cada fórmula se acompaña de la función de Julia que la realiza, tomando como referencia principal la versión con interfaz gráfica nativa (`lineal_regresion_guinativa.jl`).

2.1. El modelo

El modelo lineal univariado predice la salida como una función afín de la entrada:

$$f_{(w,b)}(x) = wx + b \tag{1}$$

donde w es la pendiente y b el intercepto. En la analogía de la radio, w es la perilla principal y b el ajuste fino. En el código esto es una línea:

```
function modelo(w, b, x)
    return w * x + b
```

```
end
```

Se escribe como función independiente para reutilizarla en la función de costo, en las derivadas y en la graficación, sin repetir la fórmula.

2.2. La función de costo

Para medir el error del modelo se usa el error cuadrático medio con un factor $\frac{1}{2}$:

$$J(w,b) = \frac{1}{2m} \sum_{i=1}^{m} \left(f_{(w,b)}\left(x^{(i)}\right) - y^{(i)} \right)^2 \tag{2}$$

Elevar al cuadrado evita que errores positivos y negativos se cancelen y penaliza más los errores grandes. El factor $\frac{1}{2}$ es conveniencia: al derivar, el 2 del exponente se cancela con el denominador, dejando expresiones más limpias. En el código:

```
function funcion_costo(w, b, x_datos, y_datos)
    m = length(x_datos)
    suma = sum((modelo(w, b, x_datos[i]) - y_datos[i])^2 for i in 1:m
        )
    return suma / (2.0 * m)
end
```

La línea del `sum` es una transcripción casi literal de (2): para cada ejemplo i se calcula el error, se eleva al cuadrado y se suma. Esta claridad es característica de Julia: el bucle explícito no tiene penalización de rendimiento porque el compilador JIT lo traduce a instrucciones de máquina nativas [2].

2.3. Las derivadas parciales

Para decidir hacia dónde mover w y b se necesita la pendiente local del costo, es decir, el gradiente:

$$\frac{\partial J}{\partial w} = \frac{1}{m} \sum_{i=1}^{m} \left(\hat{y}^{(i)} - y^{(i)} \right) x^{(i)} \tag{3}$$

$$\frac{\partial J}{\partial b} = \frac{1}{m} \sum_{i=1}^{m} \left(\hat{y}^{(i)} - y^{(i)} \right) \tag{4}$$

La única diferencia entre ambas es el factor $x^{(i)}$ en la de w, que aparece porque $\partial(wx+b)/\partial w = x$. Para b, como $\partial(wx+b)/\partial b = 1$, solo se promedian los errores. Las dos funciones de Julia son espejo de las fórmulas:

```
function dj_dw(w, b, x_datos, y_datos)
    m = length(x_datos)
    suma = sum((modelo(w,b,x_datos[i]) - y_datos[i]) * x_datos[i] for
        i in 1:m)
```

```
    return suma / m
end

function dj_db(w, b, x_datos, y_datos)
    m = length(x_datos)
    suma = sum(modelo(w,b,x_datos[i]) - y_datos[i] for i in 1:m)
    return suma / m
end
```

En la analogía, estas son las dos flechas que el oído interno produce: una para saber cómo girar la perilla grande (w) y otra para ajustar la antena (b).

2.4. Gradiente descendiente con actualización simultánea

El algoritmo se mueve en dirección contraria al gradiente con paso α:

$$w \leftarrow w - \alpha \frac{\partial J}{\partial w} \tag{5}$$

$$b \leftarrow b - \alpha \frac{\partial J}{\partial b} \tag{6}$$

Hay un detalle importante: ambas actualizaciones deben usar los mismos valores de w y b en el momento de calcular las derivadas. Si se actualiza w primero y luego se usa ese w nuevo para calcular $\partial J/\partial b$, el resultado es incorrecto. La solución, denominada *swap* en las notas, es calcular variables temporales antes de asignar:

```
for iter in 1:num_iter
    grad_w = dj_dw(w, b, x_datos, y_datos)
    grad_b = dj_db(w, b, x_datos, y_datos)

    temp_w = w - alfa * grad_w
    temp_b = b - alfa * grad_b
    w = temp_w
    b = temp_b

    costo = funcion_costo(w, b, x_datos, y_datos)
    push!(hist_costo, costo)
    push!(hist_w, w)
    push!(hist_b, b)
end
```

En cada iteración se guardan los valores de w, b y J en tres vectores de historial. Estos historiales son esenciales para las gráficas: permiten reconstruir el camino que siguió el optimizador y dibujar la parábola de costo con la secuencia de puntos por los que pasó el algoritmo.

2.5. Estabilidad numérica y elección de α

Durante el desarrollo se observó que con datasets de rango grande (por ejemplo $x \in [0, 100]$) el valor por defecto $\alpha = 0{,}01$ producía `NaN` en pocas iteraciones. La causa es directa: como $\partial J/\partial w$ incluye $x^{(i)}$, cuando los x son grandes los pasos se vuelven gigantes y el algoritmo diverge. La condición teórica de convergencia es $\alpha < 2/\mathbb{E}[x^2]$ [3]. La tabla 1 resume los valores seguros para los datasets considerados.

Cuadro 1: Tasa de aprendizaje máxima segura por dataset.

Dataset	**Rango de x**	**α máximo**
3–7 puntos manuales	[1, 7]	$\approx 0{,}08$
20 puntos aleatorios	[0, 20]	$\approx 0{,}015$
50 puntos aleatorios	[0, 50]	$\approx 0{,}0024$
100 puntos aleatorios	[0, 100]	$\approx 0{,}0006$

Por eso la versión final usa datasets pequeños con $x \in [1, 7]$, donde $\alpha = 0{,}01$ e iteraciones $= 100$ convergen sin problemas. En la analogía: si la mano que gira la perilla es muy brusca (α alto) para ondas de señal grandes, la estación se pasa siempre de largo.

2.6. Versión Gtk nativa (`lineal_regresion_guinativa.jl`)

La primera implementación construye una ventana nativa con la biblioteca `Gtk.jl`. El archivo está organizado de abajo hacia arriba en orden de abstracción creciente: primero los datos constantes, luego `modelo`, `funcion_costo`, `dj_dw`, `dj_db`, después `gradiente_descendiente`, luego `graficar`, y finalmente `lanzar_aplicacion` que es la GUI. Cada función solo usa funciones definidas antes.

La interfaz tiene cuatro campos de parámetros (`alfa`, iteraciones, paso de muestreo de rectas, máximo de rectas en la gráfica de evolución), cuatro botones con datasets predefinidos de 3, 5, 6 y 7 puntos, una sección de entrada manual (dos campos de texto con valores separados por coma) y un panel de resultados. Al presionar cualquier botón se ejecuta el ciclo: leer parámetros, llamar a `gradiente_descendiente`, calcular el costo final, mostrarlo en el panel y llamar a `graficar`.

Las gráficas se muestran en el navegador usando el backend `plotlyjs()`, que permite zoom, desplazamiento y tooltips con los valores exactos bajo el cursor. La figura final tiene cuatro paneles en cuadrícula 2×2:

1. **Regresión ajustada:** los puntos originales y la recta final $f^*(x) = w^*x + b^*$.
2. **Evolución de las rectas:** un abanico de rectas intermedias tomadas cada `paso_recta` iteraciones, en tonos de azul que pasan de claro (primeras) a oscuro (últimas), con la recta óptima superpuesta en rojo discontinuo.
3. **Parábola de costo:** $J(w, b^*)$ como función de w, con los puntos naranja que marcan la posición de w en cada iteración del algoritmo y un diamante verde en el mínimo. Esta es la superficie de costo.

4. **Convergencia:** $J(w, b)$ contra el número de iteración, con la típica curva de decaimiento exponencial que baja rápido al inicio y se aplana.

Es importante no confundir los paneles 3 y 4. El panel 4 muestra cómo el costo cambia *en el tiempo* (el proceso); el panel 3 muestra la *forma geométrica* de la función de costo (el espacio). El panel 3 es la razón visual por la que el algoritmo funciona: la superficie tiene forma de tazón convexo, sin mínimos locales, y bajar siempre en dirección del gradiente necesariamente lleva al fondo.

2.7. Versión Pluto reactiva (`regresion_pluto.jl`)

La segunda implementación es un cuaderno de `Pluto.jl` [5] que reimplementa las mismas funciones matemáticas, pero reemplaza toda la GUI por widgets `@bind`:

```
md"""**learning␣rate␣(alfa):**␣$(@bind␣alfa
␣␣␣␣␣NumberField(0.0001:0.0001:1.0,␣default=0.01))"""
md"""**numero␣de␣iteraciones:**␣$(@bind␣iters
␣␣␣␣␣NumberField(10:10:10000,␣default=100))"""
md"""**conjunto␣de␣datos:**␣$(@bind␣ds
␣␣␣␣␣Select(["3 puntos","5 puntos","6 puntos",
␣␣␣␣␣␣␣␣␣␣␣␣␣"7 puntos","manual"],␣default="5 puntos"))"""
```

A diferencia de Jupyter, Pluto es estrictamente reactivo: cambiar cualquier control dispara automáticamente la re-ejecución de todas las celdas que dependen de ese valor, sin necesidad de presionar ningún botón. Así, mover el `NumberField` de α re-entrena el modelo y redibuja las cuatro gráficas al instante, lo que permite observar de primera mano cómo una tasa de aprendizaje muy alta hace que el optimizador rebote sobre la parábola, o cómo una muy baja lo deja a medio camino después de 100 iteraciones.

La lógica matemática es idéntica a la de la versión Gtk: mismas funciones `modelo`, `funcion_costo`, `dj_dw`, `dj_db` y `gradiente_descendiente`. Lo que cambia es el envoltorio: unas 150 líneas de código GUI en Gtk se convierten en cero líneas de GUI en Pluto, reemplazadas por seis directivas `@bind`. El cuaderno también usa `import PlotlyJS` (en lugar de `using`) para evitar un conflicto de nombres con `Plots.scatter`.

2.8. Versión con biblioteca (`regresion_lineal_libreria.jl`)

La tercera implementación es solo de contraste. Resuelve el problema en una sola llamada a `GLM.jl`:

```
using GLM, DataFrames
function ajustar_modelo(x_datos, y_datos)
    df = DataFrame(X = x_datos, Y = y_datos)
    return lm(@formula(Y ~ X), df)
end
```

Internamente resuelve las ecuaciones normales $\boldsymbol{\theta}^* = (\mathbf{X}^\top\mathbf{X})^{-1}\mathbf{X}^\top\mathbf{y}$ mediante factorización QR, obteniendo la solución óptima exacta en una sola operación matricial. No requiere α ni número

de iteraciones porque no es iterativa.

Esta versión no pretende enseñar el algoritmo, sino mostrar cómo se trabaja con bibliotecas: el usuario ya no decide los hiperparámetros, estos están resueltos internamente por quienes implementaron `GLM.jl`. En ese sentido, *usar una biblioteca es casi equivalente a aceptar un conjunto de hiperparámetros ya validados*: el estudiante no toca α ni las iteraciones porque alguien más ya hizo el trabajo de elegir una estrategia numéricamente estable. De ahí el valor de haber implementado primero la versión manual: quien entiende el código de `lineal_regresion_guinativa.jl` puede leer después la documentación de `GLM.jl` sabiendo exactamente qué abstrae la biblioteca.

3. Resultados

Los tres programas producen resultados consistentes entre sí. Con los datasets manuales en $x \in [1, 7]$ y $\alpha = 0{,}01$, 100 iteraciones, el gradiente descendiente converge a pendientes cercanas a 1,3–1,5 e interceptos cercanos a 0,3–0,7, según el dataset. La versión con GLM sobre los mismos puntos devuelve valores ligeramente distintos pero dentro del mismo orden, confirmando que ambos métodos encuentran esencialmente la misma recta.

En la versión Gtk, el panel de parábola de costo muestra con claridad cómo los puntos naranjas, inicialmente espaciados en la parte alta de la curva, se acumulan en el vado cerca del mínimo a medida que avanzan las iteraciones. En la versión Pluto, mover el campo de α entre 0,005 y 0,08 (el máximo seguro aproximado para este rango de datos) basta para reproducir el fenómeno completo, desde una convergencia lenta hasta oscilaciones previas al colapso numérico.

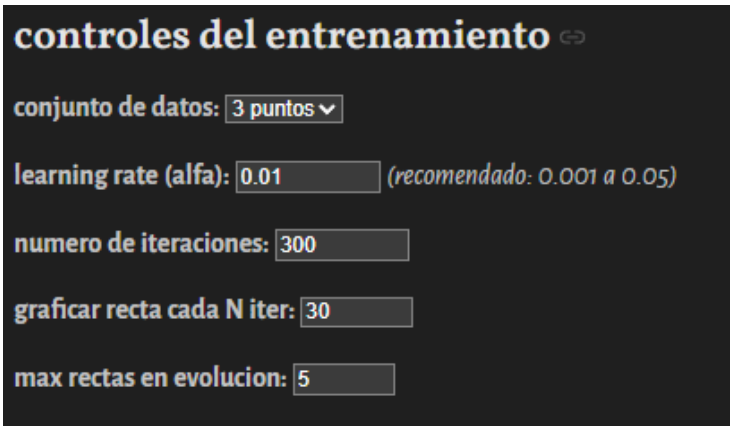
El abanico de rectas del panel 2 muestra que las primeras iteraciones producen rectas casi horizontales cerca del origen (porque w y b arrancan en cero) y que la pendiente y el intercepto se aproximan progresivamente a sus valores óptimos, acumulándose al final cerca de la recta roja discontinua que marca la solución.

3.1. Cuatro experimentos ilustrativos

Para hacer concretos los conceptos anteriores, se ejecutaron cuatro corridas del programa `regresion_pluto.jl` con configuraciones elegidas para exponer comportamientos opuestos: dos casos de convergencia exitosa y dos de fallo del optimizador. En cada figura, la captura superior muestra los parámetros ingresados; las dos imágenes inferiores muestran, a la izquierda, la recta ajustada y la superficie de costo $J(w)$, y a la derecha, la evolución de las rectas intermedias y la curva de convergencia.

Caso 1 — Convergencia rápida (3 puntos, $\alpha = 0{,}01$, 300 iteraciones)

Con un dataset pequeño y una tasa moderada el algoritmo no tiene dificultad. La curva de costo desciende de forma casi exponencial en las primeras cincuenta iteraciones y luego se aplana. En la analogía de la radio, es girar la perilla con pulso firme: pocos movimientos y casi todo el ruido desaparece. Los puntos naranjas en la parábola ilustran el recorrido de w desde el origen hasta el mínimo, donde la derivada se anula y el algoritmo deja de moverse.



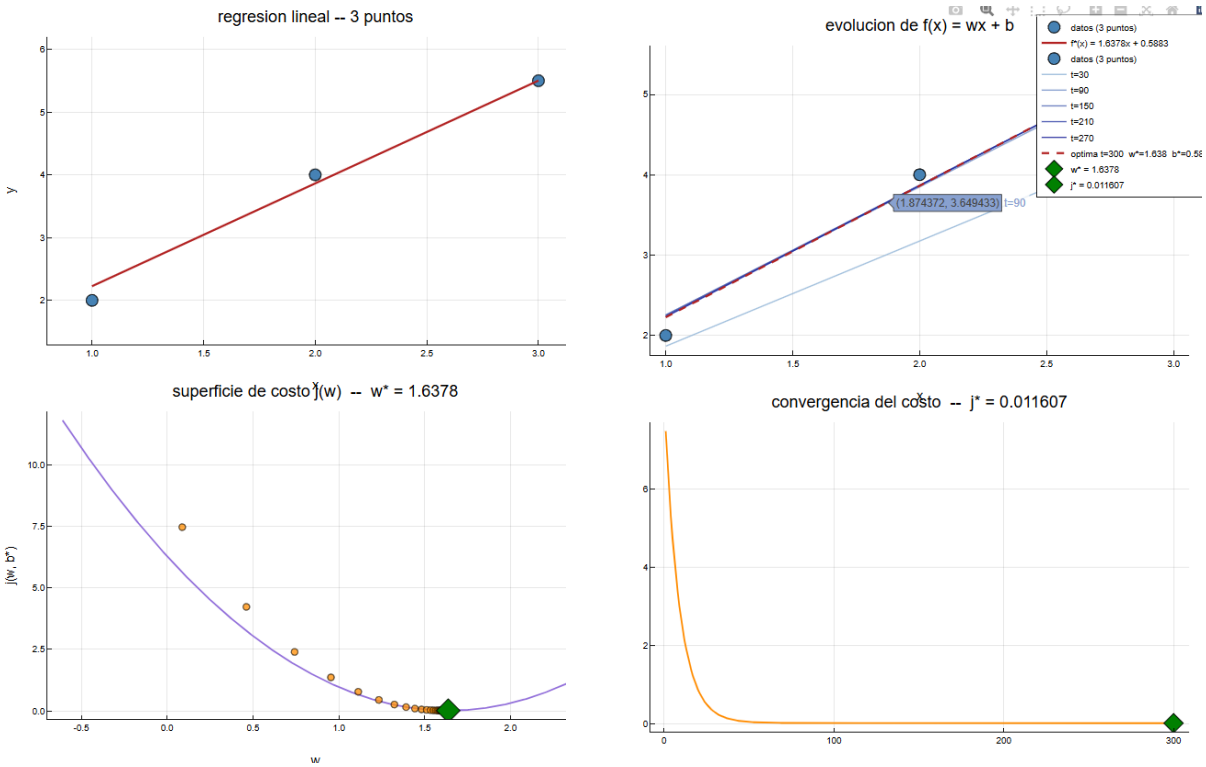


Figura 1: Caso 1: convergencia exitosa con 3 puntos, $\alpha = 0{,}01$ y 300 iteraciones. La recta ajustada pasa cerca de los tres puntos. En la parábola los puntos naranjas se acumulan en el mínimo (diamante verde, $w^* \approx 1{,}64$). La curva de convergencia baja rápido y se aplana, indicando que el algoritmo llegó al fondo con costo final $J^* \approx 0{,}012$.

Caso 2 — Convergencia gradual (7 puntos, $\alpha = 0{,}003$, 300 iteraciones)

Al ampliar el dataset a 7 puntos, el gradiente $\partial J/\partial w$ acumula términos multiplicados por valores de x hasta 7, lo que agranda los pasos implícitamente. Reducir α de 0,01 a 0,003 compensa ese efecto [3]. La convergencia es igualmente correcta pero más lenta; la recta final se ajusta bien a los siete puntos y la parábola muestra el mismo patrón de descenso escalonado hasta el mínimo.

Caso 3 — Divergencia por α excesivo (7 puntos, $\alpha = 0{,}2$, 50 iteraciones)

Este es el caso de la mano demasiado brusca. Con $\alpha = 0{,}2$ cada actualización de w produce un salto que sobrepasa el mínimo; el siguiente salto es aún mayor en la dirección contraria, y

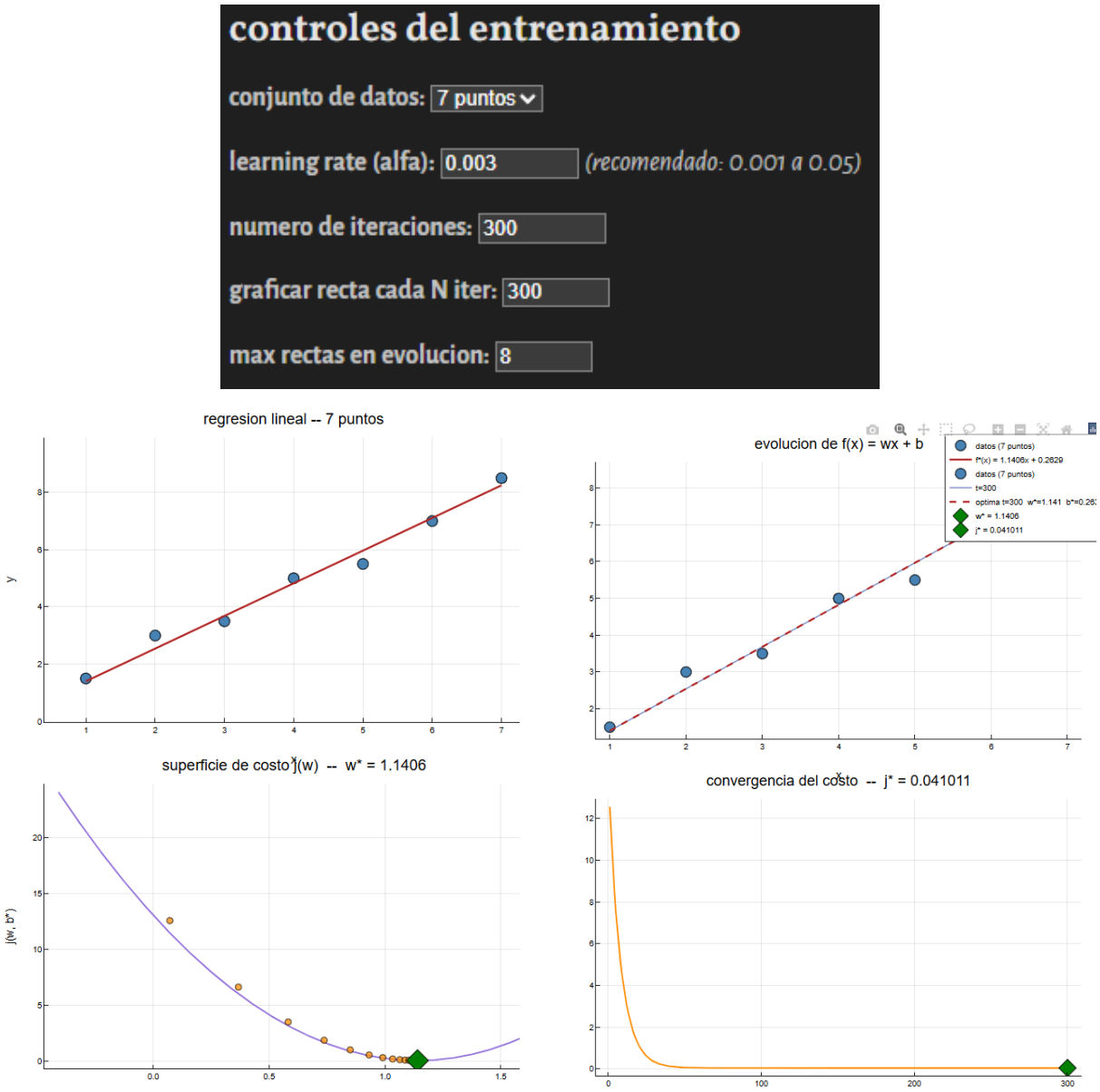


Figura 2: Caso 2: convergencia exitosa con 7 puntos, $\alpha = 0{,}003$ y 300 iteraciones. La recta cubre bien los siete puntos y el mínimo queda en $w^* \approx 1{,}14$. Los pasos son más pequeños que en el caso 1 porque α es menor, pero el destino es el mismo: el fondo de la parábola.

la secuencia diverge. Las gráficas lo confirman: los ejes escalan hasta 10^{25}, los datos originales quedan aplastados en $y \approx 0$ a esa escala, y la curva de costo explota en las últimas iteraciones en lugar de bajar. En la analogía, la mano es tan brusca que la perilla pasa la estación, rebota al otro lado, cada vez más lejos.

Caso 4 — Iteraciones insuficientes (5 puntos, $\alpha = 0{,}005$, 8 iteraciones)

Aquí el problema no es la tasa de aprendizaje sino la paciencia. Con solo 8 iteraciones el algoritmo apenas comienza a bajar por la parábola: el diamante verde queda a mitad de la ladera ($w \approx 0{,}50$), la recta resultante es casi horizontal y falla en acercarse a los datos. La curva de convergencia es la señal definitiva: sigue cayendo en la última iteración sin ningún indicio de haberse aplanado.

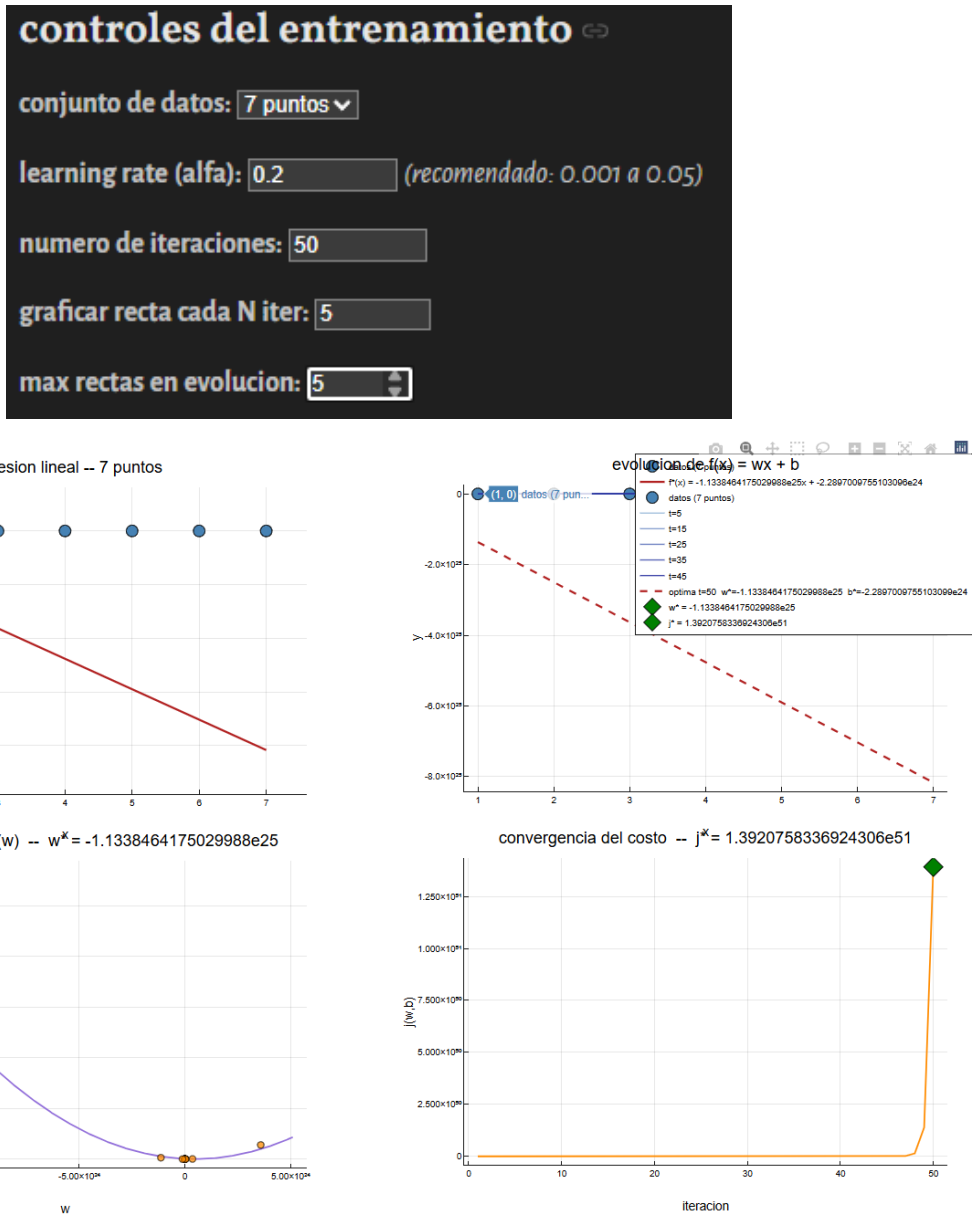


Figura 3: Caso 3: divergencia con 7 puntos, $\alpha = 0{,}2$ y 50 iteraciones. Los datos originales quedan invisibles porque el eje y escala hasta 10^{25}. La curva de convergencia permanece casi plana durante 40 iteraciones y luego explota hasta $J^* \approx 1{,}39 \times 10^{51}$; los valores finales son $w^* \approx -1{,}13 \times 10^{25}$, sin significado físico.

En la analogía, es darle ocho clics a la perilla, notar que la estática baja, y apagar el aparato antes de llegar a la estación.

4. Análisis y Discusión

El ejercicio hace visible algo que las explicaciones puramente matemáticas suelen dejar en el abstracto: el gradiente descendiente es, literalmente, un procedimiento para rodar hacia abajo por una superficie convexa. El panel 3 de la gráfica es una prueba visual de ese hecho. La parábola existe, tiene un único mínimo, y los puntos naranja bajan por ella; no hay misterio ni magia.

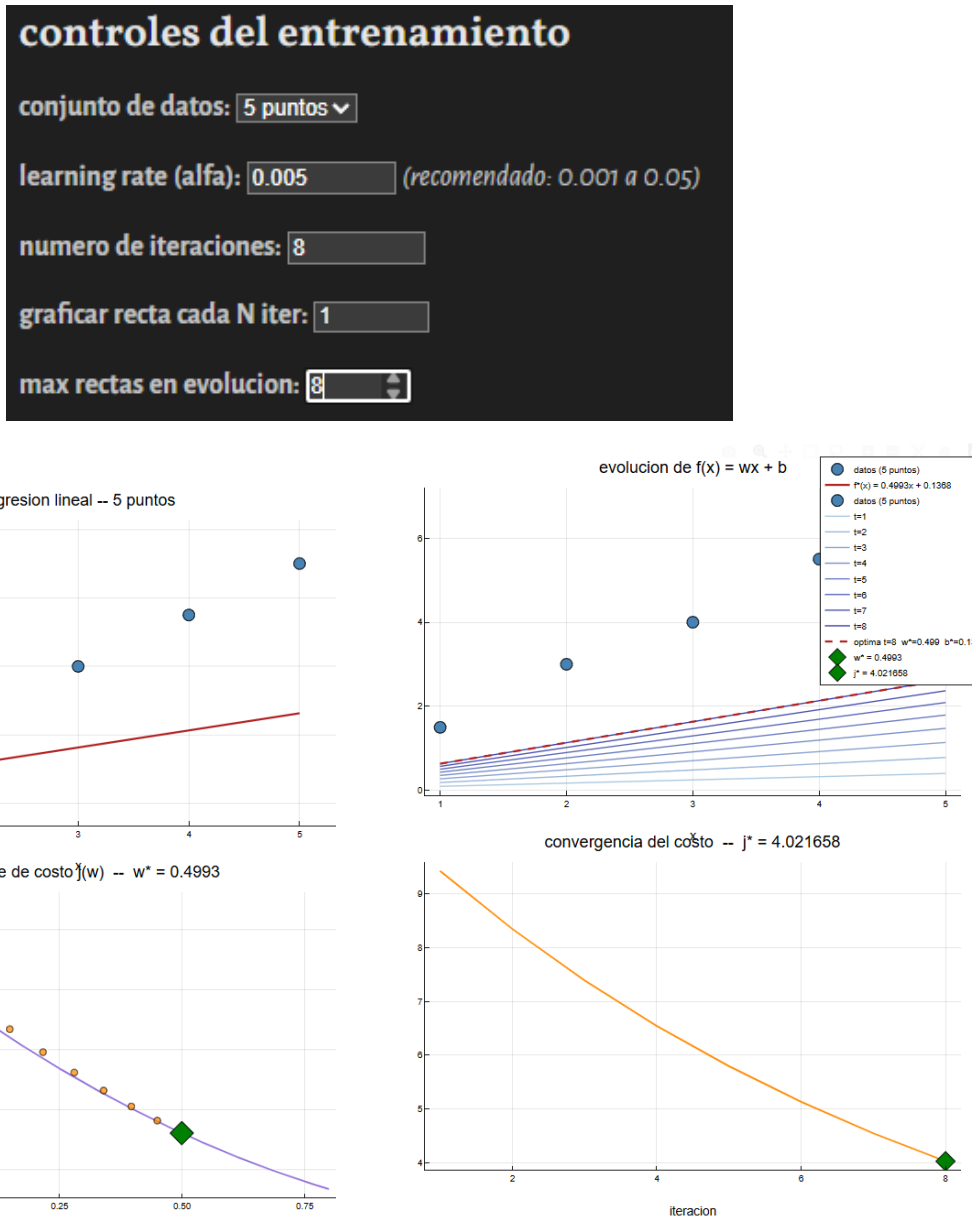


Figura 4: Caso 4: convergencia incompleta con 5 puntos, $\alpha = 0{,}005$ y 8 iteraciones. La recta queda casi horizontal y por debajo de todos los datos ($w^* \approx 0{,}50$ en lugar del óptimo $\approx 1{,}4$). El diamante verde aparece a mitad de la ladera de la parábola y la curva de costo sigue descendiendo en la última iteración: el algoritmo se detuvo antes de llegar.

La eleccción de Julia como lenguaje no es incidental. El compilador JIT especializado por tipos permite escribir bucles como `sum((modelo(w,b,x[i]) - y[i])*x[i] for i in 1:m)`, transcripción casi textual de la fórmula, sin la penalización de rendimiento que este código tendría en Python puro [2]. En Python habría que vectorizar con NumPy y reescribir el algoritmo en términos matriciales, alejándolo de la fórmula original. En Julia el estudiante puede leer el código como lee la derivada.

La comparación entre las tres versiones ilumina la economía de las bibliotecas. La versión manual en Gtk tiene alrededor de 500 líneas; la versión Pluto reduce la GUI a casi nada y queda en torno a 300 líneas; la versión con GLM cabe en menos de 100 líneas porque el trabajo lo hace `lm`. Pero

la reducción de líneas oculta decisiones: qué algoritmo numérico usar, cómo evitar divergencia, qué tolerancia aceptar. Esas decisiones son los hiperparámetros que el usuario de la biblioteca no ve. En la versión manual, se ven todos.

El costo es que la versión manual puede explotar: con datasets de rango amplio y α mal elegido, aparecen `NaN`. La biblioteca no tiene ese problema. Ambas cosas son ciertas y el contraste es el aprendizaje.

Desde la perspectiva del aprendizaje automático más amplio, el valor del gradiente descendiente no está en este caso particular —donde OLS es más rápido y más estable— sino en que es el mismo método que entrena redes neuronales, modelos logísticos y transformadores. Lo que cambia es el tamaño del modelo y la función de costo, no la lógica iterativa [4]. Implementarlo a mano en el caso más simple es una inversión con rendimiento desproporcionado: se entiende el esqueleto sobre el que se apoya el resto de la disciplina.

5. Conclusiones

El trabajo entrega dos implementaciones funcionales y una referencia. `lineal_regresion_guinativa.jl` ofrece una experiencia de escritorio con entrenamiento bajo demanda y gráficas interactivas en el navegador; `regresion_pluto.jl` replica la misma matemática en un cuaderno reactivo donde cualquier cambio de parámetro recalcula todo automáticamente; `regresion_lineal_libreria.jl` resuelve el problema con `GLM.jl` y sirve para ver que usar bibliotecas es, en la práctica, adoptar un conjunto de decisiones ya tomadas.

La analogía de la radio sostiene todo el recorrido: w es la perilla, b la antena, J el dolor de cabeza por la estática, α la brusquedad de la mano, y el gradiente el oído que indica hacia dónde girar. Al final, el estudiante debería poder leer las cuatro gráficas, asociar cada una con un paso del algoritmo, y entender por qué esa misma estructura aparece después en modelos mucho más grandes. Esa traducción directa entre fórmula, código e imagen es el aporte principal del ejercicio.

Referencias

[1] A. Ng, *Machine Learning Specialization*. Coursera / DeepLearning.AI, 2022. [En línea]. Disponible en: https://www.coursera.org/specializations/machine-learning-introduction

[2] J. Bezanson, A. Edelman, S. Karpinski, y V. B. Shah, "Julia: A fresh approach to numerical computing," *SIAM Review*, vol. 59, no. 1, pp. 65–98, 2017.

[3] T. Hastie, R. Tibshirani, y J. Friedman, *The Elements of Statistical Learning*, 2nd ed. New York, NY, USA: Springer, 2009.

[4] I. Goodfellow, Y. Bengio, y A. Courville, *Deep Learning*. Cambridge, MA, USA: MIT Press, 2016.

[5] F. van der Plas *et al.*, *Pluto.jl: simple reactive notebooks for Julia*, 2024. [En línea]. Disponible en: https://plutojl.org

[6] D. Bates *et al.*, *GLM.jl: Generalized linear models in Julia*, Julia Statistics, 2024. [En línea]. Disponible en: https://github.com/JuliaStats/GLM.jl